

Statistical Language Modeling

Chapter 4 · From counts to next-word probabilities

Dr. A. Belcaid

Table of contents

1 Predicting the next word **3**
1.1 Greedy choice versus sampling 4

2 From the chain rule to n-grams **5**
2.1 The Markov approximation 5

3 Counting one visible corpus **6**
3.1 Tokenization creates observable transitions 6
3.2 Maximum-likelihood estimation 7
3.3 A direct implementation 8

4 Generating and scoring sequences **9**
4.1 Greedy generation 9
4.2 Sampling generation 9
4.3 Scoring a complete sentence 10
4.4 Log-probabilities prevent numerical underflow 10

5 The zero-frequency problem **10**

6 Smoothing: reserve mass for the unseen **11**
6.1 Add-one smoothing 11
6.2 What happens to matrix sparsity? 12
6.3 Better smoothing families 13

7 Evaluating a language model **13**
7.1 Average negative log-probability 14
7.2 Perplexity 14
7.3 A fully visible calculation 15
7.4 Perplexity is intrinsic, not final 15

8 Why n-grams are not enough **16**
8.1 A bigram remembers one token 16
8.2 Increasing the order makes the table explode 16
8.3 Similar words cannot share counts 17

9 The question carried into recurrent networks **17**

10 Chapter synthesis **18**

11 Further reading **18**
11.1 References 19

When a phone proposes the next word, a search engine completes a query, or a speech recognizer compares possible transcriptions, the system needs a distribution over what may come next. A **language model** supplies that distribution.

This chapter develops the smallest useful language model from first principles. We will keep one corpus visible, inspect every count, and follow the model through prediction, generation, smoothing, and evaluation. Its failures will then create the question that motivates recurrent neural networks.

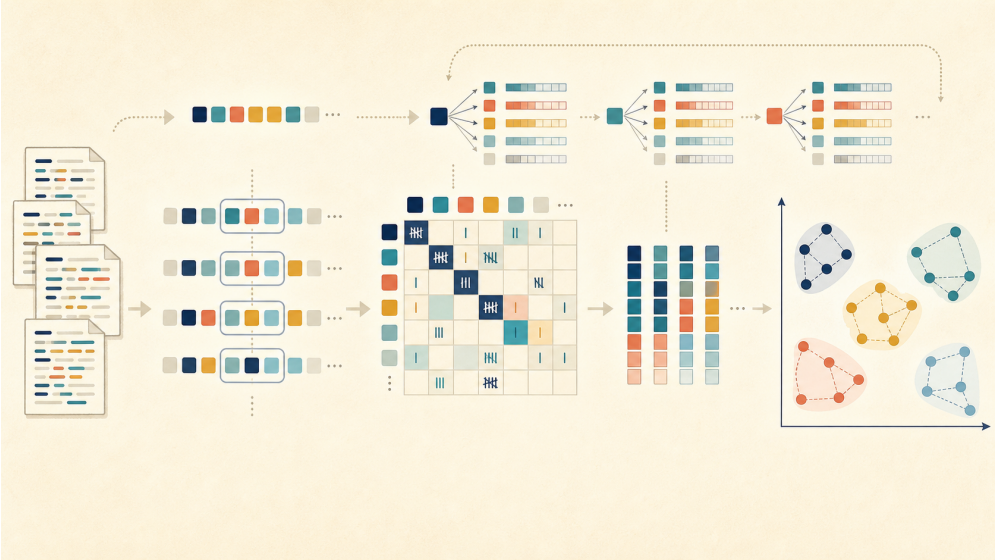


Figure 1: The lecture map: corpus evidence becomes sequences, contexts, a count matrix, and structured representations.



Figure 2: The chapter follows one dependency chain: each stage requires the object constructed by the previous stage.

Learning objectives

After this chapter, you should be able to:

1. distinguish a language-model distribution from a single prediction;
2. derive unigram, bigram, and general n-gram approximations from the chain rule;
3. construct bigram counts and maximum-likelihood probabilities from a tokenized corpus;
4. explain greedy decoding and probabilistic sampling;
5. calculate the probability and log-probability of a sequence;
6. explain why unseen n-grams produce zero probabilities and how smoothing responds;
7. calculate and interpret perplexity on held-out text; and
8. identify the memory, sparsity, and parameter-sharing limitations that motivate recurrent models.

1 Predicting the next word

Suppose the visible history is:

```
students study ...
```

Several continuations are grammatical, but they are not equally likely. A language model should not return only a word. It should return a **probability distribution**:

Table 1: A next-word distribution represents both a preferred continuation and the remaining uncertainty.

Candidate w	$P(w \mid \text{students study})$
language	0.40
statistics	0.30
together	0.15
other words	0.15

The values in Table 1 are nonnegative and sum to one. Prediction is a decision made **from** this distribution; the distribution itself is the model output (Jurafsky and Martin 2026, ch. 3).

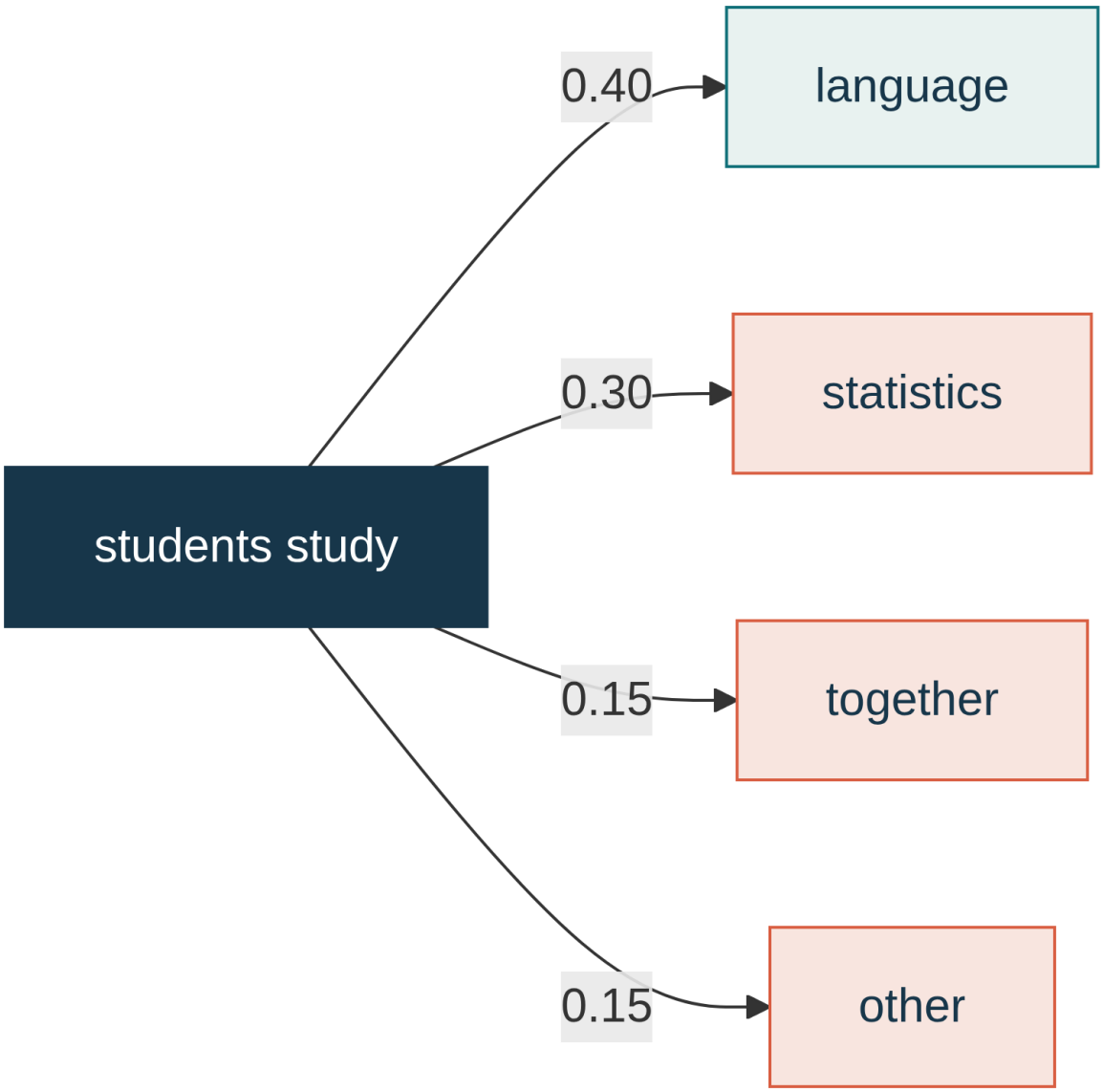


Figure 3: One history can support several continuations. A decision rule converts the distribution into an action.

1.1 Greedy choice versus sampling

Two common decision rules use the same distribution differently.

Greedy decoding selects the most probable word:

$$\hat{w} = \arg\max_{w \in V} P(w \mid h). \tag{1}$$

For Table 1, greedy decoding always selects *language*.

Sampling draws a word according to the entire categorical distribution. One implementation partitions $[0, 1)$ into cumulative intervals:

```
language     [0.00, 0.40)
statistics   [0.40, 0.70)
together     [0.70, 0.85)
other        [0.85, 1.00)
```

If $r = 0.58$, the sampled word is *statistics*. Sampling can therefore choose a less likely continuation without pretending it was the most likely one.

Prediction is not generation

Prediction performs one next-word decision. **Generation** repeats prediction, appends the selected word, and uses the enlarged sequence as the next history.

2 From the chain rule to n-grams

A language model assigns a probability to a sequence $w_{1:n} = (w_1, \dots, w_n)$. Probability theory gives an exact factorization:

$$P(w_{1:n}) = \prod_{t=1}^{n} P(w_t \mid w_{1:t-1}). \tag{2}$$

For a three-word sentence,

$$P(w_1, w_2, w_3) = P(w_1)P(w_2 \mid w_1)P(w_3 \mid w_1, w_2).$$

The chain rule is exact. The difficulty is estimating conditionals with long histories: most complete histories appear rarely or never, even in large corpora (Jurafsky and Martin 2026, ch. 3).

2.1 The Markov approximation

An n-gram model deliberately keeps only a fixed suffix of the history:

$$P(w_t \mid w_{1:t-1}) \approx P(w_t \mid w_{t-n+1:t-1}). \tag{3}$$

Table 2: Increasing the order retains more context but demands evidence for more specific histories.

Model	Retained history	Example
Unigram	none	$P(language)$
Bigram	one token	$P(language \mid study)$
Trigram	two tokens	$P(language \mid students, study)$

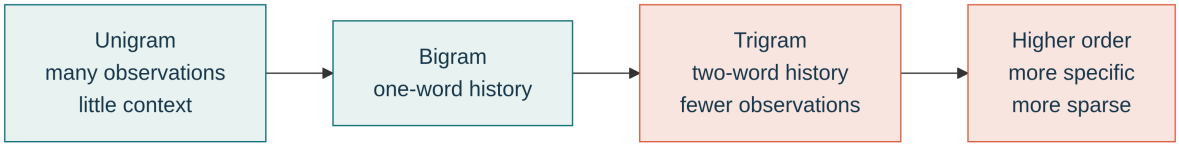


Figure 4: Higher-order n-grams gain specificity while losing observations per history.

The important trade-off is not simply "higher order is better." More context can make a prediction more specific **when enough evidence exists**, but every extra history token divides the data into smaller groups.

3 Counting one visible corpus

We now construct a bigram model from a six-sentence corpus. Boundary symbols make sentence starts and ends explicit.

```
S1  <s> students study language </s>
S2  <s> students study statistics </s>
S3  <s> researchers study language </s>
S4  <s> students learn statistics </s>
S5  <s> researchers study statistics </s>
S6  <s> students study language </s>
```

The repeated S1/S6 sentence is intentional: frequency is the training signal.

3.1 Tokenization creates observable transitions

For S1, the bigram events are:

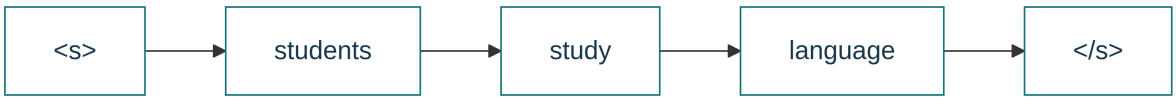


Figure 5: A tokenized sentence of length five contributes four adjacent bigram events.

Each arrow adds one to a count $C(h, w)$, where h is the history and w is the continuation.

Table 3: Nonzero bigram counts obtained from the six-sentence corpus.

History h	Continuation w	$C(h, w)$
<s>	students	4
<s>	researchers	2
students	study	3
students	learn	1
researchers	study	2

History h	Continuation w	$C(h, w)$
study	language	3
study	statistics	2
learn	statistics	1
language	`</s>`	3
statistics	`</s>`	3

The history count is the total number of transitions leaving that history:

$$C(h) = \sum_{w \in V} C(h, w). \tag{4}$$

For example,

$$C(students) = C(students, study) + C(students, learn) = 3 + 1 = 4.$$

3.2 Maximum-likelihood estimation

The relative-frequency or maximum-likelihood estimate is

$$P_{\text{MLE}}(w \mid h) = \frac{C(h, w)}{C(h)}. \tag{5}$$

The numerator counts the requested transition. The denominator counts every occasion on which the history had an opportunity to transition to something.

For the history `study`,

$$P(language \mid study) = \frac{3}{5} = 0.60, \qquad P(statistics \mid study) = \frac{2}{5} = 0.40.$$

These values form a complete row:

$$\sum_{w} P(w \mid study) = 0.60 + 0.40 = 1.$$

> ⚠ **Common denominator error**
>
> Do not divide $C(study, language)$ by the total number of corpus tokens. A conditional probability row is normalized by the number of occurrences of its **history**, $C(study)$.

Checkpoint · recover the counts first

Using the visible corpus, calculate:

1. $P(study \mid students)$;
2. $P(language \mid study)$; and
3. $P(statistics \mid study)$.

For every result, write $C(h, w)$ and $C(h)$ before dividing.

Worked solution

$$P(study \mid students) = \frac{C(students, study)}{C(students)} = \frac{3}{4} = 0.75.$$

$$P(language \mid study) = \frac{C(study, language)}{C(study)} = \frac{3}{5} = 0.60.$$

$$P(statistics \mid study) = \frac{C(study, statistics)}{C(study)} = \frac{2}{5} = 0.40.$$

3.3 A direct implementation

The implementation should preserve the same inspectable objects as the calculation.

```
from collections import Counter, defaultdict

sentences = [
    ["students", "study", "language"],
    ["students", "study", "statistics"],
    ["researchers", "study", "language"],
    ["students", "learn", "statistics"],
    ["researchers", "study", "statistics"],
    ["students", "study", "language"],
]

bigram_counts = defaultdict(Counter)
history_counts = Counter()

for sentence in sentences:
    tokens = ["<s>", *sentence, "</s>"]
    for history, word in zip(tokens, tokens[1:]):
        bigram_counts[history][word] += 1
        history_counts[history] += 1

def mle_probability(word: str, history: str) -> float:
    numerator = bigram_counts[history][word]
    denominator = history_counts[history]
```

```
    return numerator / denominator if denominator else 0.0
```

This code stores a sparse map instead of allocating every possible vocabulary pair. That choice matters because a vocabulary of size $|V|$ permits $|V|^2$ bigram cells.

4 Generating and scoring sequences

Once a probability row exists, generation is prediction in a loop:

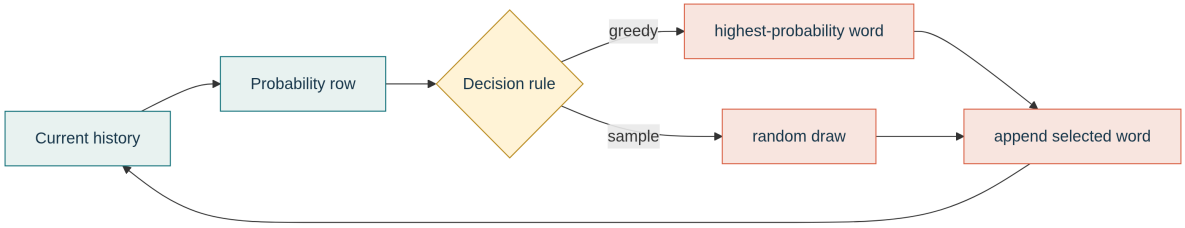


Figure 6: Autoregressive generation repeatedly selects a word, appends it, and treats the result as the next history.

4.1 Greedy generation

Starting at `<s>`, a greedy decoder follows the largest value in every row:

```
<s> --students (.67)--> students --study (.75)--> study
    --language (.60)--> language --</s> (1.00)--> stop
```

With fixed probabilities and no ties, repeating greedy decoding from the same start state produces the same sentence every time.

4.2 Sampling generation

A sampler may follow the same first steps but choose `statistics` from the `study` row with probability 0.40:

```
<s> → students → study → statistics → </s>
```

Sampling increases variety, but it also permits lower-probability and occasionally poor paths. Temperature and top-k/top-p methods refine this trade-off in modern neural generation; the fundamental distinction remains whether we maximize or draw from the distribution (Jurafsky and Martin 2026, ch. 3).

4.3 Scoring a complete sentence

The bigram approximation assigns

$$P(w_{1:n}) \approx P(w_1 \mid \langle s \rangle) \prod_{t=2}^{n} P(w_t \mid w_{t-1}) P(\langle /s \rangle \mid w_n). \tag{6}$$

For `students study language`, our corpus gives

$$\begin{aligned} P(\text{students study language}) &= P(students \mid \langle s \rangle) P(study \mid students) \\ &\quad \times P(language \mid study) P(\langle /s \rangle \mid language) \\ &= \frac{4}{6} \times \frac{3}{4} \times \frac{3}{5} \times \frac{3}{3} \\ &= 0.30. \end{aligned}$$

Longer sequences multiply more numbers below one, so their raw probabilities naturally become smaller.

4.4 Log-probabilities prevent numerical underflow

Multiplying hundreds of small floating-point values can underflow toward zero. Taking logarithms converts products into sums:

$$\log P(w_{1:n}) = \sum_{t=1}^{n} \log P(w_t \mid h_t). \tag{7}$$

Because logarithm is monotonic, the sequence with the largest probability also has the largest log-probability.

5 The zero-frequency problem

Consider a separate, deliberately limited training sample in which `study language` never appears. We are asked to score the perfectly understandable clause:

```
students study language
```

The problematic factor is visible:

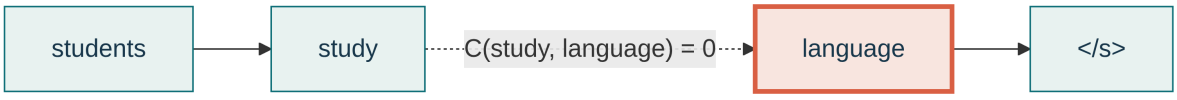


Figure 7: A single unseen transition inserts a zero into the product, regardless of the plausibility of the complete clause.

Maximum likelihood assigns

$$P_{\text{MLE}}(language \mid study) = 0,$$

and therefore the complete product is zero.

> **!** **The central diagnosis**
>
> **Unseen in a finite corpus does not mean impossible in the language.** The model has confused absence of evidence with evidence of impossibility.

This is the **zero-frequency problem**. It is particularly severe for higher-order n-grams because specific histories are less likely to repeat (Jurafsky and Martin 2026, ch. 3).

6 Smoothing: reserve mass for the unseen

Smoothing modifies the maximum-likelihood distribution so that observed events surrender a little probability mass to unseen events. The result remains normalized but no longer assigns zero to every missing transition. Extensive comparisons of smoothing methods show that the choice matters considerably for language modeling (Chen and Goodman 1999).

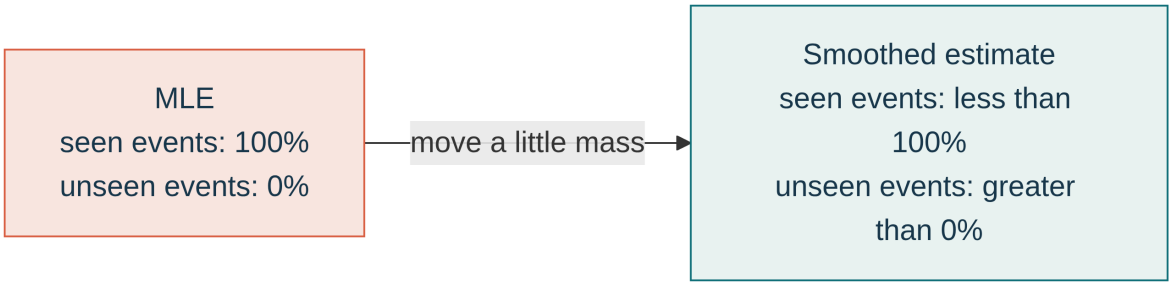


Figure 8: Smoothing does not create new observations; it redistributes estimated probability mass.

6.1 Add-one smoothing

The clearest introductory rule adds one pseudo-count to every candidate:

$$P_{+1}(w \mid h) = \frac{C(h, w) + 1}{C(h) + |V|}. \tag{8}$$

The denominator adds $|V|$ because one pseudo-count was added to each of the $|V|$ possible continuations.

If $C(study) = 4$, $C(study, together) = 0$, and $|V| = 4$, then

$$P_{+1}(together \mid study) = \frac{0 + 1}{4 + 4} = 0.125.$$

Add-one smoothing makes the logic inspectable, but for large word vocabularies it usually removes too much mass from observed events. It is therefore mainly a teaching baseline, not a strong practical word-level language model (Jurafsky and Martin 2026, ch. 3; Chen and Goodman 1999).

6.2 What happens to matrix sparsity?

Consider four possible continuations. The unsmoothed probability matrix contains six zero cells:

Table 4: A sparse maximum-likelihood probability matrix.

$P_{\text{MLE}}(w \mid h)$	language	statistics	together	</s>
study	.60	.40	**0**	**0**
learn	.50	**0**	.50	**0**
model	**0**	.33	**0**	.67

After add-one smoothing, all twelve cells are positive:

Table 5: The corresponding dense smoothed probability matrix; bold entries were zero before smoothing.

$P_{+1}(w \mid h)$	language	statistics	together	</s>
study	.44	.33	**.11**	**.11**
learn	.33	**.17**	.33	**.17**
model	**.14**	.29	**.14**	.43

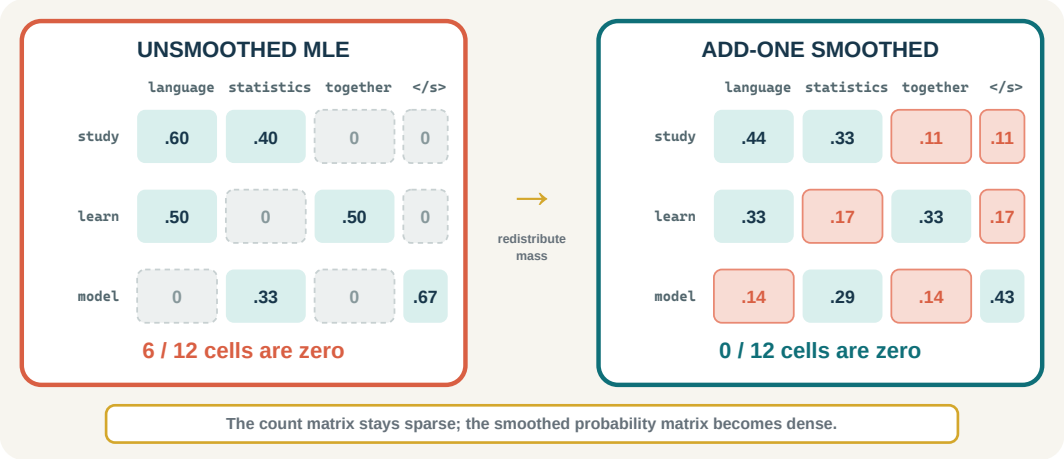


Figure 9: Smoothing redistributes probability mass from observed transitions to formerly zero cells.

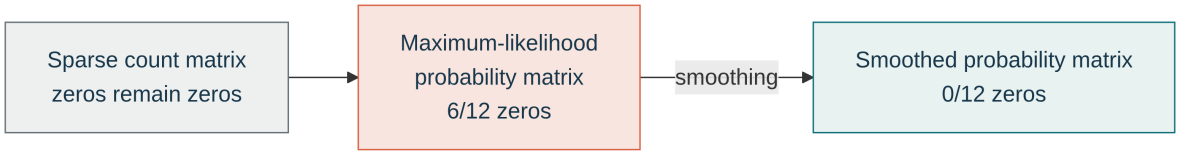


Figure 10: The observations do not change. Only the estimated probability table becomes dense.

> ⚠ **Precise wording matters**
>
> The **count matrix remains sparse**: smoothing does not hallucinate corpus events. The **estimated probability matrix becomes dense** because formerly unseen events receive small positive probabilities.

6.3 Better smoothing families

Practical n-gram models use the structure of the count distribution more carefully.

- **Backoff** uses a lower-order model when the detailed history has weak or missing evidence. Katz's influential method combines discounting with backoff (Katz 1987).
- **Interpolation** mixes estimates from several orders instead of selecting only one.
- **Kneser–Ney smoothing** discounts observed counts and uses a continuation distribution that rewards words appearing after many distinct histories (Kneser and Ney 1995).

For example, an interpolated trigram model may use

$$P(w_t \mid w_{t-2}, w_{t-1}) = \lambda_3 P_{\text{tri}}(w_t \mid w_{t-2}, w_{t-1}) + \lambda_2 P_{\text{bi}}(w_t \mid w_{t-1}) + \lambda_1 P_{\text{uni}}(w_t),$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$.

7 Evaluating a language model

Training probability only tells us how well a model remembers its training corpus. Evaluation asks whether it generalizes to **held-out text** that was not used to estimate the counts.

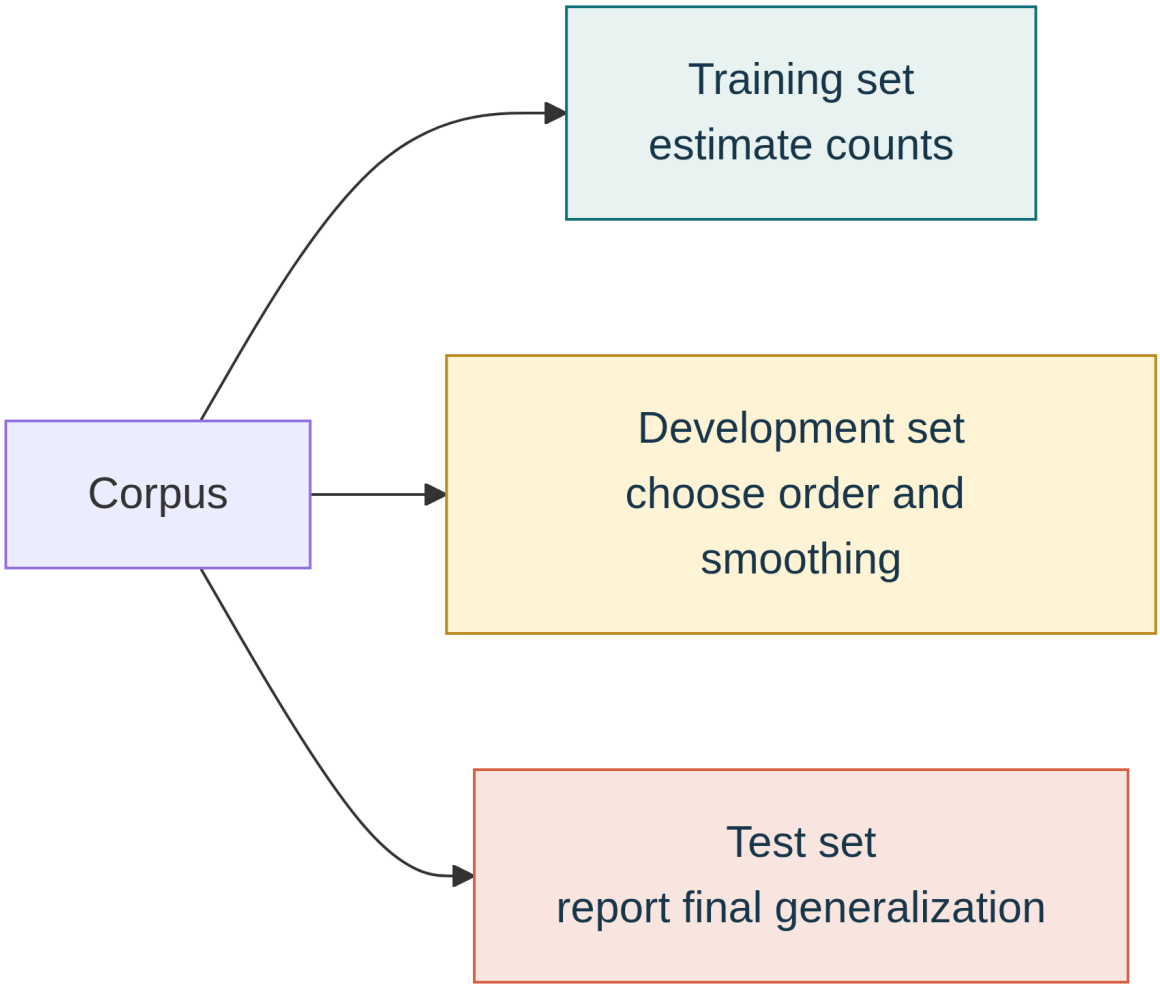


Figure 11: The test set must remain untouched until the model and its choices are fixed.

7.1 Average negative log-probability

For a test sequence of N predicted tokens, the average negative log-probability is

$$H(W) = -\frac{1}{N}\sum_{t=1}^{N} \log_2 P(w_t \mid h_t). \tag{9}$$

This quantity measures average surprise in bits. A confident correct prediction contributes little surprise; assigning a tiny probability to the observed word contributes much more.

7.2 Perplexity

Perplexity exponentiates the average surprise:

$$\mathrm{PP}(W) = 2^{H(W)} = P(w_{1:N})^{-1/N}. \tag{10}$$

Equivalently, using natural logarithms,

$$\mathrm{PP}(W) = \exp\left(-\frac{1}{N}\sum_{t=1}^{N}\log P(w_t \mid h_t)\right).$$

Lower perplexity means the model assigns more probability, on average, to the observed held-out tokens (Jurafsky and Martin 2026, ch. 3).

7.3 A fully visible calculation

Suppose a test sequence produces four conditional probabilities:

$$\frac{1}{2}, \quad \frac{1}{4}, \quad \frac{1}{2}, \quad 1.$$

Their product is

$$P(W) = \frac{1}{2} \times \frac{1}{4} \times \frac{1}{2} \times 1 = \frac{1}{16}.$$

With $N = 4$ predicted tokens,

$$\mathrm{PP}(W) = \left(\frac{1}{16}\right)^{-1/4} = 16^{1/4} = 2.$$

The model behaves as though it faces two equally plausible choices at each position on average. This “effective branching factor” is an intuition, not a literal claim that every row contains exactly two words.

Checkpoint · compare two models

Model A has test perplexity 2.1 and Model B has test perplexity 3.4. Which model better predicts the test text? What additional information is required before the comparison is valid?

Worked answer

Model A has the lower average held-out surprise. The comparison is valid only if both values use the same test text, tokenization, vocabulary, unknown-word policy, and treatment of boundary tokens. Perplexities computed under different tokenizations are not directly comparable.

7.4 Perplexity is intrinsic, not final

Perplexity measures the language model itself. An application ultimately needs an **extrinsic** metric:

Table 6: Intrinsic improvements are valuable only when they support the real task.

Application	Example downstream measure
Autocomplete	suggestion acceptance rate
Speech recognition	word error rate
Machine translation	human judgments or translation-quality metric
Text generation	task success, factuality, safety, and human preference

8 Why n-grams are not enough

N-gram models are useful precisely because their assumptions are visible. The same visibility makes their limitations easy to diagnose.

8.1 A bigram remembers one token

Consider:

> The **keys** to the cabinet ___

English agreement favors *are* because the subject *keys* is plural. A bigram model sees only `cabinet → ?`; the relevant cue has disappeared.

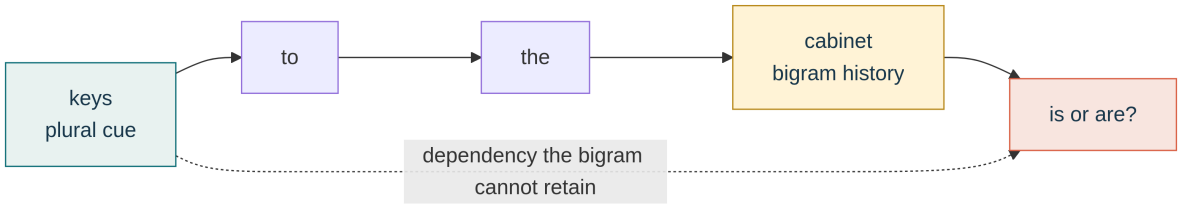


Figure 12: The predictive cue can lie outside a fixed n-gram window.

8.2 Increasing the order makes the table explode

With a vocabulary of $50{,}000$ tokens:

$$|V|^2 = 2.5 \text{ billion possible bigrams},$$

$$|V|^3 = 125 \text{ trillion possible trigrams}.$$

Most possible n-grams are never observed. Raising n extends memory but worsens parameter growth and sparsity.

8.3 Similar words cannot share counts

Suppose the corpus contains:

```
students   → study    observed 12 times
researchers → study   observed 9 times
pupils     → study    observed 0 times
```

Even if *students* and *pupils* are semantically similar, the n-gram table treats their transitions as unrelated entries. Evidence collected for one symbol does not automatically improve the other.

Table 7: Four pressures that motivate learned, distributed sequence representations.

Limitation	Observable symptom
Fixed memory	distant cues disappear
Parameter growth	$\lvert V\rvert^n$ possible n-grams
Sparsity	most entries remain unobserved
No distributed sharing	similar words use separate counts

9 The question carried into recurrent networks

We do not solve the next problem here. We only make its desired structure visible.

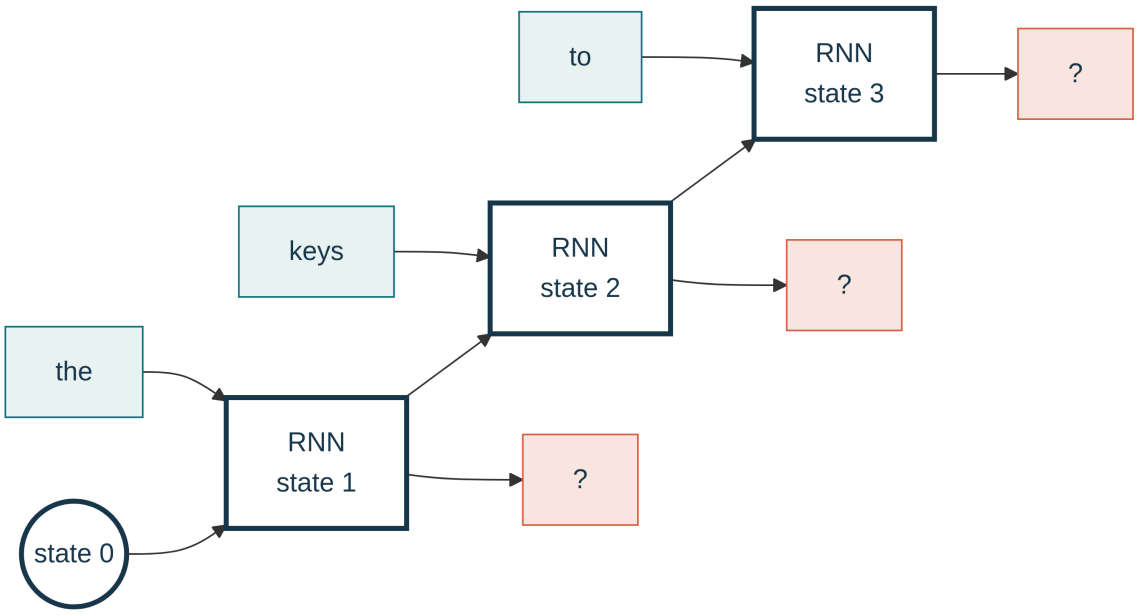


Figure 13: A classical unrolled recurrent network reuses one kind of state update through time. The next chapter asks what the state contains and how it learns.

The diagram promises a state that moves through time, but leaves the central questions unresolved:

- How is the state updated?
- What information survives many steps?
- How are words represented so that similar words can share evidence?
- How does the final state become a next-word distribution?

> **! Question for the next lecture**
>
> If a model carries one learned state through an arbitrarily long history, **how can it learn what deserves to remain in memory?**

10 Chapter synthesis

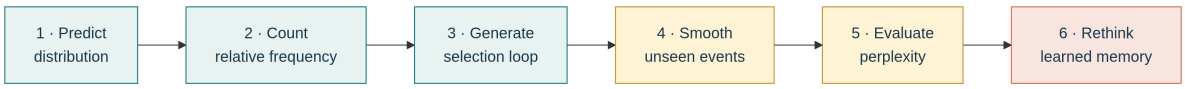


Figure 14: The complete statistical language-modeling journey.

The core ideas are compact:

1. A language model maps a history to a probability distribution over the next token.
2. The chain rule is exact; the n-gram history restriction is an approximation.
3. Maximum-likelihood bigram probabilities are continuation counts divided by history counts.
4. Generation repeats prediction, using either maximization or sampling.
5. One unseen factor collapses an unsmoothed sequence probability to zero.
6. Smoothing reserves probability for unseen events and makes the estimated probability matrix dense.
7. Perplexity measures average held-out surprise under carefully matched evaluation conditions.
8. Fixed histories, explosive tables, sparsity, and isolated symbols motivate learned recurrent state.

11 Further reading

- (Jurafsky and Martin 2026, ch. 3) gives the main modern treatment of n-gram language models, smoothing, and perplexity used throughout this chapter.
- Shannon (1948) established the information-theoretic concepts behind entropy and sequence prediction.
- Chen and Goodman (1999) provides a systematic empirical comparison of smoothing techniques.
- Katz (1987) develops an influential discounting and backoff estimator for sparse language data.
- Kneser and Ney (1995) introduces the continuation-based idea behind Kneser–Ney smoothing.

11.1 References

Chen, Stanley F., and Joshua Goodman. 1999. “An Empirical Study of Smoothing Techniques for Language Modeling.” *Computer Speech & Language* 13 (4): 359–94. https://doi.org/10.1006/csla.1999.0128.

Jurafsky, Daniel, and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd ed. https://web.stanford.edu/~jurafsky/slp3/.

Katz, Slava M. 1987. “Estimation of Probabilities from Sparse Data for the Language Model Component of a Speech Recognizer.” *IEEE Transactions on Acoustics, Speech, and Signal Processing* 35 (3): 400–401. https://doi.org/10.1109/TASSP.1987.1165125.

Kneser, Reinhard, and Hermann Ney. 1995. “Improved Backing-Off for M-Gram Language Modeling.” *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* 1: 181–84. https://doi.org/10.1109/ICASSP.1995.479394.

Shannon, Claude E. 1948. “A Mathematical Theory of Communication.” *The Bell System Technical Journal* 27 (3): 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x.